MLA02 – FUNDAMENTALS OF MACHINE LEARNING
QUESTION 4

Artificial Neural Network Using Backpropagation

Introduction

A multilayer perceptron makes predictions in the forward pass and updates weights using backpropagation to reduce prediction error. These examples use scikit-learn's MLPClassifier.

Implementation library / technique: MLPClassifier (feed-forward neural network trained using backpropagation).

General workflow: prepare the data, separate features and target labels, train the model, classify a new sample, and display the result.

Problem 1: Student Result Prediction

Aim

To implement Artificial Neural Network Using Backpropagation for student result prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes StudyHours, Attendance, Assignments and their class labels to train a model, then classify a new sample.

Algorithm

1. Import the required Python libraries.
2. Store the training samples in a feature matrix X and labels in y.
3. Create the classifier and fit it using X and y.
4. Prepare the new sample and apply the same preprocessing, if required.
5. Predict and display the class of the new sample.

Input (Training Data)

StudyHours	Attendance	Assignments	Target
1	50	0	0
2	55	0	0
3	60	1	0
4	70	1	1
5	75	1	1
6	80	1	1
2	45	0	0
5	90	1	1
3	65	0	0
7	85	1	1

Program

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['StudyHours', 'Attendance', 'Assignments']
training_data = [[1, 50, 0, 0], [2, 55, 0, 0], [3, 60, 1, 0], [4, 70, 1, 1], [5, 75, 1, 1], [6, 80, 1, 1], [2, 45, 0, 0], [5, 90, 1, 1], [3, 65, 0, 0], [7, 85, 1, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = MLPClassifier(hidden_layer_sizes=(8,), activation="relu",
                      solver="lbfgs", max_iter=2000, random_state=42)
model.fit(X_scaled, y)
```

```
new_sample = np.array([4, 75, 1], dtype=float)
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Artificial Neural Network Using Backpropagation was implemented for student result prediction and the new sample was classified using the trained model.

Problem 2: Customer Purchase Prediction

Aim

To implement Artificial Neural Network Using Backpropagation for customer purchase prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Visits, TimeOnSite, DiscountSeen and their class labels to train a model, then classify a new sample.

Algorithm

6. Import the required Python libraries.
7. Store the training samples in a feature matrix X and labels in y.
8. Create the classifier and fit it using X and y.
9. Prepare the new sample and apply the same preprocessing, if required.
10. Predict and display the class of the new sample.

Input (Training Data)

Visits	TimeOnSite	DiscountSeen	Target
1	2	0	0
2	3	0	0
3	5	1	1
4	6	1	1
1	1	0	0
5	7	1	1
2	4	1	0
6	8	1	1
3	3	0	0
5	6	0	1

Program

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['Visits', 'TimeOnSite', 'DiscountSeen']
training_data = [[1, 2, 0, 0], [2, 3, 0, 0], [3, 5, 1, 1], [4, 6, 1, 1], [1, 1, 0, 0], [5, 7, 1, 1], [2, 4, 1, 0], [6, 8, 1, 1], [3, 3, 0, 0], [5, 6, 0, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = MLPClassifier(hidden_layer_sizes=(8,), activation="relu",
                      solver="lbfgs", max_iter=2000, random_state=42)
model.fit(X_scaled, y)
```

```
new_sample = np.array([4, 6, 1], dtype=float)
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Artificial Neural Network Using Backpropagation was implemented for customer purchase prediction and the new sample was classified using the trained model.

Problem 3: Machine Fault Detection

Aim

To implement Artificial Neural Network Using Backpropagation for machine fault detection and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes Temperature, Vibration, OperatingHours and their class labels to train a model, then classify a new sample.

Algorithm

11. Import the required Python libraries.
12. Store the training samples in a feature matrix X and labels in y.
13. Create the classifier and fit it using X and y.
14. Prepare the new sample and apply the same preprocessing, if required.
15. Predict and display the class of the new sample.

Input (Training Data)

Temperature	Vibration	OperatingHours	Target
30	1	10	0
35	1	20	0
70	8	80	1
75	9	90	1
40	2	25	0
80	10	100	1
45	3	30	0
68	7	75	1
32	2	15	0
78	8	95	1

Program

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['Temperature', 'Vibration', 'OperatingHours']
training_data = [[30, 1, 10, 0], [35, 1, 20, 0], [70, 8, 80, 1], [75, 9, 90, 1], [40, 2, 25, 0], [80, 10, 100, 1], [45, 3, 30, 0], [68, 7, 75, 1], [32, 2, 15, 0], [78, 8, 95, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = MLPClassifier(hidden_layer_sizes=(8,), activation="relu",
                      solver="lbfgs", max_iter=2000, random_state=42)
model.fit(X_scaled, y)
```

```
new_sample = np.array([72, 8, 85], dtype=float)
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Artificial Neural Network Using Backpropagation was implemented for machine fault detection and the new sample was classified using the trained model.

Problem 4: Iris Flower Class Prediction

Aim

To implement Artificial Neural Network Using Backpropagation for iris flower class prediction and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes SepalLength, SepalWidth, PetalLength and their class labels to train a model, then classify a new sample.

Algorithm

16. Import the required Python libraries.
17. Store the training samples in a feature matrix X and labels in y.
18. Create the classifier and fit it using X and y.
19. Prepare the new sample and apply the same preprocessing, if required.
20. Predict and display the class of the new sample.

Input (Training Data)

SepalLength	SepalWidth	PetalLength	Target
5.1	3.5	1.4	0
4.9	3.0	1.4	0
5.0	3.4	1.5	0
6.0	2.7	4.5	1
5.9	3.0	4.2	1
6.1	2.8	4.7	1
6.5	3.0	5.5	2
6.7	3.1	5.6	2
6.3	2.9	5.4	2
5.8	2.7	4.1	1

Program

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['SepalLength', 'SepalWidth', 'PetalLength']
training_data = [[5.1, 3.5, 1.4, 0], [4.9, 3.0, 1.4, 0], [5.0, 3.4, 1.5, 0], [6.0, 2.7, 4.5, 1], [5.9, 3.0, 4.2,
1], [6.1, 2.8, 4.7, 1], [6.5, 3.0, 5.5, 2], [6.7, 3.1, 5.6, 2], [6.3, 2.9, 5.4, 2], [5.8, 2.7, 4.1, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = MLPClassifier(hidden_layer_sizes=(8,), activation="relu",
                      solver="lbfgs", max_iter=2000, random_state=42)
model.fit(X_scaled, y)
```

```
new_sample = np.array([5.2, 3.4, 1.5], dtype=float)
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Artificial Neural Network Using Backpropagation was implemented for iris flower class prediction and the new sample was classified using the trained model.

Problem 5: Energy Usage Category

Aim

To implement Artificial Neural Network Using Backpropagation for energy usage category and predict the class of a new sample.

Problem Statement

Use the given training examples with attributes PeopleAtHome, ApplianceCount, Temperature and their class labels to train a model, then classify a new sample.

Algorithm

21. Import the required Python libraries.
22. Store the training samples in a feature matrix X and labels in y.
23. Create the classifier and fit it using X and y.
24. Prepare the new sample and apply the same preprocessing, if required.
25. Predict and display the class of the new sample.

Input (Training Data)

PeopleAtHome	ApplianceCount	Temperature	Target
1	2	18	0
2	3	20	0
3	5	24	1
4	6	28	1
1	1	17	0
5	7	30	1
2	4	22	0
4	5	26	1
3	4	21	0
5	8	32	1

Program

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
import numpy as np

attributes = ['PeopleAtHome', 'ApplianceCount', 'Temperature']
training_data = [[1, 2, 18, 0], [2, 3, 20, 0], [3, 5, 24, 1], [4, 6, 28, 1], [1, 1, 17, 0], [5, 7, 30, 1], [2, 4, 22, 0], [4, 5, 26, 1], [3, 4, 21, 0], [5, 8, 32, 1]]
X = np.array([row[:-1] for row in training_data], dtype=float)
y = np.array([row[-1] for row in training_data])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
model = MLPClassifier(hidden_layer_sizes=(8,), activation="relu",
                      solver="lbfgs", max_iter=2000, random_state=42)
model.fit(X_scaled, y)
```

```
new_sample = np.array([4, 6, 27], dtype=float)
prediction = model.predict(scaler.transform(new_sample))
print("Predicted class:", prediction[0])
```

Output

The program displays the predicted class for the supplied new sample. The exact predicted label is printed when the program is run in Python with scikit-learn installed.

Result

Thus, Artificial Neural Network Using Backpropagation was implemented for energy usage category and the new sample was classified using the trained model.

Conclusion

Five different application problems were demonstrated using Artificial Neural Network Using Backpropagation. The results show how a trained machine-learning model can use example data to classify a new observation. Performance depends on the quality and size of the training data.

Requirements

Python 3, scikit-learn, NumPy, and a Python IDE or Jupyter Notebook.